Parameters

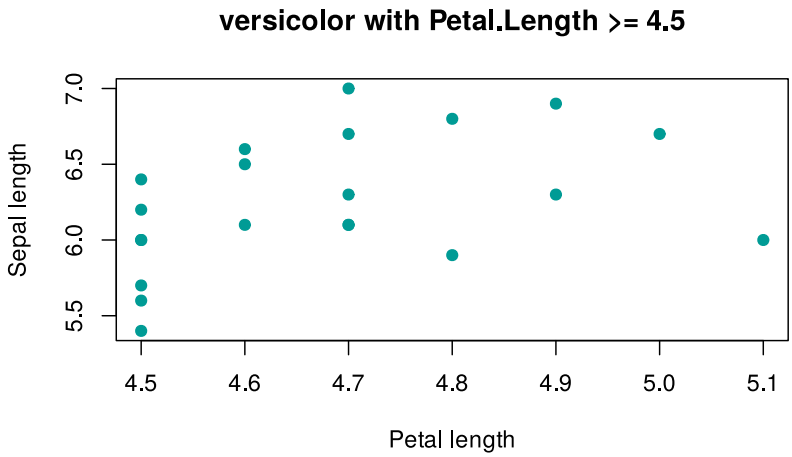
Variables let you render the same notebook with different inputs. Declare them in `#calepin.setup`, and chunks read them through a `vars` object. For the `r` and `python` engines, *Calepin* creates the binding automatically: R receives a named `list` read as `vars$Species`, and Python receives a `dict` read as `vars["Species"]`.

This complete document filters the built-in R `iris` data to one species and a minimum petal length, then uses a third parameter to choose the color palette:

````
#import "/.calepin/calepin.typ" as calepin

#calepin.setup(
  vars: (
    Species: "versicolor",
    min_petal_length: 4.5,
    palette: "viridis",
  ),
)

The selected species is #calepin.inline("r")[`cat(vars$Species)`].

```r
#| fig-caption: Iris rows selected by document variables
filtered <- subset(
  iris,
  Species == vars$Species & Petal.Length >= vars$min_petal_length
)

colors <- hcl.colors(3, palette = vars$palette)

plot(
  Sepal.Length ~ Petal.Length,
  data = filtered,
  pch = 19,
  col = colors[2],
  xlab = "Petal length",
  ylab = "Sepal length",
  main = paste(vars$Species, "with Petal.Length >=", vars$min_petal_length)
)
```
````

The selected species is versicolor.

```r
filtered <- subset(
  iris,
  Species == vars$Species & Petal.Length >= vars$min_petal_length
)

colors <- hcl.colors(3, palette = vars$palette)

plot(
  Sepal.Length ~ Petal.Length,
  data = filtered,
  pch = 19,
  col = colors[2],
  xlab = "Petal length",
  ylab = "Sepal length",
  main = paste(vars$Species, "with Petal.Length >=", vars$min_petal_length)
)
```



Overriding at render time

Because variables live in `#calepin.setup`, you can override them on the command line with `--var key=value` (repeatable). This renders the same source with different inputs, without editing the notebook:

```
calepin compile iris.typ --var Species=setosa --var min_petal_length=1.5 --var palette=magma
```

Command-line values are typed the same way as `#|` header values, so `1.5` is a number, `true` is a boolean, and `setosa` is a string.

Value types

A variable may be `none`, a boolean, an integer, a float, a string, an array, or a dictionary, nested freely. Other Typst values such as content, functions, lengths, colors, and dates cannot be passed directly; supply them as strings or numbers instead. An unsupported value fails the build with a message naming the offending variable.

Variables are also written to `.calepin/<document>/vars.json`. Engines reached through a Jupyter kernel, including `julia` and any other kernel, do not yet receive an automatic `vars` binding. Read that JSON file yourself using the `CALEPIN_VARS_PATH` environment variable that *Calepin* sets in the kernel. Variables are not secret: treat them as build inputs written to disk.